LAB SESSION 9

Lab Name: Coding, Implementation & Version Control

Project: MediScript-E — Digital Healthcare Platform

Objectives

- Implement system modules following design specifications
- Apply clean coding practices and TypeScript standards
- Use Git for version control and collaborative development

Theory

Implementation must:

- **Follow design:** Code must reflect the architecture and UML models defined in Lab Session 6
- **Follow coding standards:** TypeScript strict typing, consistent naming conventions, modular structure
- **Use version control:** Git enables tracking changes, collaboration, and rollback capability

---

Task 1: Implemented Modules

Module 1: Authentication System

Files Implemented:

- `src/lib/auth.ts` — NextAuth configuration with credentials, Google, GitHub providers + profile picture in JWT/session
- `src/lib/db.ts` — Prisma client singleton with pg adapter and SSL configuration
- `src/app/api/register/route.ts` — User registration API
- `src/app/api/verify-email/route.ts` — Email verification API
- `src/app/api/auth/2fa/send/route.ts` — 2FA OTP send API
- `src/app/api/auth/2fa/verify/route.ts` — 2FA OTP verify API
- `src/app/login/page.tsx` — Login page with credentials and OAuth
- `src/app/register/page.tsx` — Registration page
- `src/app/verify-2fa/page.tsx` — 2FA OTP verification page
- `src/components/UserAvatar.tsx` — Reusable avatar with image/initials fallback

Key Implementation — OAuth Profile Picture in Session:

```
// src/lib/auth.ts
async jwt({ token, user, account, profile }) {
  if (user) {
    token.id = user.id;
    token.role = (user as any).role || "PATIENT";
```

```
      token.image = user.image ?? null;
    } else if (account?.provider !== "credentials" && token.email) {
      const dbUser = await db.user.findUnique({ where: { email: token.email }
});
      if (dbUser) { token.id = dbUser.id; token.role = dbUser.role; }
      if (profile?.image) token.image = profile.image as string;
      else if ((profile as any)?.avatar_url) token.image = (profile as
any).avatar_url;
    }
    return token;
  },
  async session({ session, token }) {
    if (session.user) {
      session.user.id = token.id as string;
      session.user.role = token.role as Role;
      session.user.image = token.image ?? null;
    }
    return session;
  },
```

Key Implementation — UserAvatar Component:

```
// src/components/UserAvatar.tsx
export default function UserAvatar({ name, image, size, gradient }:
UserAvatarProps) {
  if (image) {
    return (
      <Image src={image} alt={name ?? "User"} width={size} height={size}
        className="rounded-full object-cover" referrerPolicy="no-referrer"
/>
    );
  }
  return (
    <div className={`rounded-full bg-gradient-to-br ${gradient} flex items-
center justify-center text-white font-black`}
      style={{ width: size, height: size }}>
      {name?.charAt(0).toUpperCase() ?? "U"}
    </div>
  );
}
```

Module 2: Professional Dashboard Structure

Files Implemented:

- `src/app/dashboard/page.tsx` — Overview with stats and quick actions
- `src/app/appointments/page.tsx` — Dedicated appointments page
- `src/app/prescriptions/page.tsx` — Dedicated prescriptions page
- `src/app/reminders/page.tsx` — Dedicated medicine reminders page

- `src/app/vault/page.tsx` — Dedicated medical vault page
- `src/app/users/page.tsx` — Admin user management page
- `src/app/contacts/page.tsx` — Admin contact messages page
- `src/app/feedback/page.tsx` — Share feedback page
- `src/components/DashboardLayout.tsx` — Layout with sidebar + header + page transitions
- `src/components/DashboardHeader.tsx` — Top header with search bar and user avatar
- `src/components/DashboardSidebar.tsx` — Sidebar with role-based navigation
- `src/components/PageTransition.tsx` — Framer Motion page transitions
- `src/components/ConditionalNavbar.tsx` — Hides navbar on dashboard routes

Key Implementation — Page Transition:

```
// src/components/PageTransition.tsx
export default function PageTransition({ children }: { children: ReactNode
}) {
  const pathname = usePathname();
  return (
    <AnimatePresence mode="wait">
      <motion.div key={pathname}
        initial={{ opacity: 0, y: 12 }}
        animate={{ opacity: 1, y: 0 }}
        exit={{ opacity: 0, y: -8 }}
        transition={{ duration: 0.2, ease: "easeOut" }}>
        {children}
      </motion.div>
    </AnimatePresence>
  );
}
```

Key Implementation — Sidebar Active State:

```
// src/components/DashboardSidebar.tsx
const isActive = (href: string, exact?: boolean) => {
  if (exact) return pathname === "/dashboard";
  return pathname === href || pathname.startsWith(href + "/");
};
```

Module 3: Appointment System

Files Implemented:

- `src/app/api/appointment/route.ts` — GET (fetch appointments), POST (create appointment)
- `src/app/api/appointment/[id]/route.ts` — PATCH (update status with ownership check), DELETE
- `src/components/BookAppointment.tsx` — Patient appointment booking UI
- `src/components/MyAppointments.tsx` — Patient appointments list with filter tabs

- `src/components/DoctorAppointments.tsx` — Doctor appointments with confirmation dialogs

Key Implementation — Appointment Ownership Authorization:

```
// src/app/api/appointment/[id]/route.ts
if (session.user.role === "DOCTOR") {
  const doctor = await db.doctorProfile.findUnique({ where: { userId:
session.user.id } });
  if (!doctor || appointment.doctorId !== doctor.id)
    return NextResponse.json({ message: "Forbidden" }, { status: 403 });
} else if (session.user.role === "PATIENT") {
  const patient = await db.patientProfile.findUnique({ where: { userId:
session.user.id } });
  if (!patient || appointment.patientId !== patient.id)
    return NextResponse.json({ message: "Forbidden" }, { status: 403 });
}
```

Module 4: Prescription System

Files Implemented:

- `src/app/api/prescription/route.ts` — POST (create + auto-complete appointment), GET
- `src/app/api/prescription/[id]/route.ts` — PATCH (edit/archive), DELETE
- `src/components/PrescriptionForm.tsx` — Doctor prescription issuance with patient dropdown
- `src/components/DoctorPrescriptionList.tsx` — Active/Archived tabs with edit/archive/delete
- `src/components/DownloadPDF.tsx` — PDF generation

Key Implementation — Auto-Complete Appointment on Prescription:

```
// src/app/api/prescription/route.ts
const newPrescription = await db.prescription.create({
  data: { diagnosis, medications, doctorId: doctor.doctorProfile.id,
patientId },
});

// Auto-complete the specific appointment
if (appointmentId) {
  await db.appointment.update({
    where: { id: appointmentId },
    data: { status: "COMPLETED" },
  });
}
```

Key Implementation — Archive Toggle:

```
// src/app/api/prescription/[id]/route.ts
const { diagnosis, medications, archivedByDoctor } = await req.json();

if (typeof archivedByDoctor === "boolean") {
  const updated = await db.prescription.update({
    where: { id },
    data: { archivedByDoctor },
  });
  return NextResponse.json({ message: "Prescription updated", prescription:
updated });
}
```

Module 5: Medicine Reminder System

Files Implemented:

- `src/app/api/medicine-reminder/route.ts` — GET, POST reminders
- `src/app/api/medicine-reminder/[id]/route.ts` — PATCH (mark taken), DELETE
- `src/app/api/medicine-reminder/send-notifications/route.ts` — Cron job endpoint
- `src/components/AddMedicineReminder.tsx` — Dynamic time inputs based on frequency
- `src/components/MedicineReminders.tsx` — Active/inactive separation

Key Implementation — Send Notifications (Cron):

```
export async function POST(req: NextRequest) {
  const apiKey = req.headers.get("x-api-key");
  if (apiKey !== process.env.CRON_API_KEY) {
    return NextResponse.json({ message: "Unauthorized" }, { status: 401 });
  }
  // Uses Bangladesh time (UTC+6) for matching
  const reminders = await db.medicineReminder.findMany({
    where: { taken: false, startDate: { lte: endOfTomorrow }, endDate: {
gte: startOfToday } },
    include: { patient: { include: { user: true } } },
  });
  for (const reminder of reminders) {
    await transporter.sendMail({ to: reminder.patient.user.email, ... });
  }
}
```

Module 6: Medical Vault

Files Implemented:

- `src/app/api/vault/route.ts` — POST (upload document)
- `src/app/api/vault/[id]/route.ts` — DELETE (delete document)

- `src/components/FileUpload.tsx` — File upload UI with Supabase Storage integration
- `src/components/RecordItem.tsx` — Individual record display component

Module 7: AI Chatbot (MediBot)

Files Implemented:

- `src/app/api/chatbot/route.ts` — Groq API integration with system prompt
- `src/components/Chatbot.tsx` — Floating chat UI with conversation history

Key Implementation — Chatbot API with Markdown Stripping:

```
// src/app/api/chatbot/route.ts
const completion = await groq.chat.completions.create({
  model: "llama-3.1-8b-instant",
  messages: [
    { role: "system", content: SYSTEM_PROMPT },
    ...messages.map((m) => ({ role: m.role, content: m.content })),
  ],
  max_tokens: 512,
});

const raw = completion.choices[0].message.content ?? "";
// Strip all markdown formatting
const reply = raw
  .replace(/\*\*(.*?)\*\*/g, "$1")
  .replace(/\*(.*?)\*/g, "$1")
  .replace(/^[*-]\s/gm, "")
  .replace(/#{1,6}\s/g, "")
  .trim();
```

Module 8: Global Search

Files Implemented:

- `src/app/api/search/route.ts` — Role-based search API
- `src/components/DashboardHeader.tsx` — Search bar with debounce and dropdown

Key Implementation — Debounced Search:

```
// src/components/DashboardHeader.tsx
useEffect(() => {
  if (!query.trim()) { setResults({}); setOpen(false); return; }
  if (debounceRef.current) clearTimeout(debounceRef.current);
  debounceRef.current = setTimeout(async () => {
    setLoading(true);
    const res = await fetch(`/api/search?q=${encodeURIComponent(query)}`);
    const data = await res.json();
```

```
      setResults(data.results || {});
      setOpen(true);
      setLoading(false);
    }, 350);
  }, [query]);
```

Module 9: Admin Dashboard

Files Implemented:

- `src/app/api/admin/stats/route.ts` — Real-time statistics
- `src/app/api/admin/users/route.ts` — User management
- `src/app/api/admin/users/[id]/route.ts` — Delete user
- `src/app/api/admin/appointments/route.ts` — All appointments
- `src/app/api/admin/contacts/route.ts` — Contact messages
- `src/components/AdminDashboard.tsx` — Admin stats component
- `src/components/UserManagement.tsx` — User management component
- `src/components/AppointmentOverview.tsx` — Appointment overview component

Module 10: Settings

Files Implemented:

- `src/app/api/settings/profile/route.ts` — Update profile name
- `src/app/api/settings/password/route.ts` — Change password
- `src/app/api/settings/2fa/route.ts` — Toggle 2FA
- `src/components/SettingsForm.tsx` — Settings UI with profile, password, and 2FA sections

---

Task 2: Git Version Control

Repository

- **GitHub Repository:** `https://github.com/tusharsno/mediscript-e`
- **Branch:** `main`
- **Deployment:** Vercel (manual `vercel --prod` CLI)

Key Commits

Commit Message	Description
`Initial project setup`	Next.js 16 project initialization
`Add authentication system`	Registration, login, email verification
`Add appointment booking`	Patient and doctor appointment management

Commit Message	Description
`Add prescription system`	Doctor prescription issuance and PDF download
`Add medicine reminders`	Reminder scheduling with cron job
`Add medical vault`	Supabase Storage file upload
`Add admin dashboard`	Real-time stats and user management
`Add 2FA email OTP verification`	Two-factor authentication feature
`feat: add MediBot AI chatbot powered by Groq`	AI chatbot integration
`feat: dashboard search bar with global search`	Global search feature
`feat: prescription archive/unarchive for doctors`	Prescription management
`feat: profile picture support for OAuth users`	OAuth profile pictures
`feat: professional dashboard restructure`	Dedicated routes per feature
`feat: page transition animations`	Framer Motion transitions

Task 3: Secure Coding Practices Applied

Practice	Implementation
Input validation	All API routes validate required fields before processing
Parameterized queries	Prisma ORM used for all database operations — no raw SQL
Password hashing	`bcrypt.hash(password, 10)` — never stored in plaintext
Environment variables	All secrets in `.env` — never hardcoded in source code
Role-based authorization	`getServerSession()` checked on every protected API route
Ownership authorization	Appointment PATCH/DELETE checks user owns the resource
TypeScript strict typing	All components and API routes use TypeScript interfaces
Selective data queries	Prisma `select` used to return only required fields
Token expiry	Verification tokens (24h), OTP (10min), JWT sessions (30 days)
Error handling	Try-catch blocks on all async operations
SSL/TLS	Database connection uses `sslmode=no-verify` for Supabase
AI content safety	System prompt restricts chatbot to platform-related responses

Practice	Implementation
Markdown stripping	AI responses stripped of markdown before display

Key Findings / Learning Outcomes

- Successfully implemented **10 major modules** with **40+ API endpoints** following the design from Lab Session 6
- Learned that **professional web apps** use dedicated routes per feature rather than single-page scroll navigation
- Understood that **OAuth profile pictures** require image domain whitelisting in Next.js config
- Recognized that **AI chatbot** requires careful system prompt design and response post-processing
- Applied **ownership authorization** on appointment endpoints to prevent cross-user data access
- Learned that **page transitions** with Framer Motion significantly improve perceived performance
- Version control enabled safe experimentation — broken changes could be reverted using `git revert`